

Database Implementation Documentation

Overview

Our backend now includes a fully functional PostgreSQL database layer that supports ingestion, storage, and retrieval of IoT sensor data. The implementation follows a clean Controller → Service → Repository architecture to ensure modularity, maintainability, and testability.

Data Ingestion Pipeline

The ingestion script retrieves IoT sensor data from our data source and stores it in PostgreSQL in wide-format rows (field1–field8).

Each ingested row includes:

- created_at (timestamp)
- entry_id (unique reading ID)
- field1 ... field8 (sensor metrics)

Ingestion automatically creates datasets and inserts all associated time-series rows.

Database Schema

datasets table

- id (primary key)
- name (e.g., “sensor1”)

timeseries table (for csv injection)

- id (primary key)
- dataset_id (foreign key → datasets.id)
- created_at
- entry_id
- field1 ... field8

timeseries table wide (for thingSpeak ingestion)

- id (primary key)
- dataset_id (foreign key → datasets.id)
- created_at (timestamp from ThingSpeak feed)
- entry_id (unique ThingSpeak reading ID)
- field1
- field2
- field3
- field4
- field5
- field6
- field7
- field8

This schema supports efficient querying and aligns with the ingestion format.

Repository Layer

Each repository handles SQL operations for a specific domain:

- DatasetRepository – retrieves dataset metadata
- SeriesRepository – retrieves wide-format time-series rows
- (Future) TimestampsRepository – for timestamp-specific queries

All repositories use a shared PostgreSQL connection pool.

Service Layer

The service layer provides business logic:

- DatasetService – fetch all datasets, fetch by name or ID
- SeriesService – return all series for a dataset, filter by metrics

This ensures controllers remain thin and focused.

API Endpoints

Dataset Endpoints • GET /api/datasets Returns all datasets stored in the database. • GET /api/datasets/:id Returns a single dataset by ID. • GET /api/datasets/:name Returns a dataset by name. • POST /api/datasets Disabled. Dataset creation is handled automatically by ingestion.

Series Endpoints • GET /api/datasets/:name/series Returns all time-series rows for the selected dataset in wide format. • POST /api/datasets/:name/series/filter Filters the dataset's series by selected metrics (e.g., field1, field3). • (Planned) GET /api/datasets/:name/series?start=&end= Supports absolute time-range filtering. • (Planned) GET /api/datasets/:name/series/recent?minutes= Supports relative time-range filtering.

Timestamp Endpoints • GET /api/datasets/:name/timestamps Returns all timestamps for the dataset. • (Planned) GET /api/datasets/:name/timestamps/latest Returns the most recent timestamp. • (Planned) GET /api/datasets/:name/timestamps/range Returns timestamps within a given range.

Analysis Endpoints • POST /api/analyse Accepts selected fields and returns analysis results (correlation, variance, etc.). • (Planned) POST /api/analyse/correlation Computes correlation between exactly two selected fields. • (Planned) POST /api/analyse/summary Returns summary statistics for selected fields.

Ingestion Endpoints • POST /api/ingest Triggers ingestion of new sensor data. • GET /api/ingest/status Returns ingestion health and last run time. • (Planned) GET /api/ingest/logs Returns ingestion logs for debugging.

Current Status

- Database connection is stable and fully functional.
- Ingestion pipeline successfully populates the database.
- Dataset and series endpoints now return real ingested data.
- Architecture is consistent across all backend modules.